

Laboratorio 1 – GPIO

Programación en Ensamblador del ATmega328P

Pablo Ramírez, Braian Arias, Angelo Ferrari
Ingeniería Mecatrónica
Tecnologías de Microprocesamiento – Semestre IV
Universidad Tecnológica del Uruguay
Docentes: Jesús Martínez y Marcelo Díaz

Resumen—En el presente laboratorio se desarrollaron cuatro aplicaciones utilizando el microcontrolador ATmega328P programado en lenguaje ensamblador AVR. El objetivo principal fue trabajar con los recursos fundamentales de la arquitectura del microcontrolador, principalmente los puertos de entrada y salida GPIO, registros de propósito general, registro de estado SREG, operaciones aritméticas y lógicas, saltos condicionales y retardos implementados mediante software.

El laboratorio estuvo compuesto por cuatro problemas. En el Problema A se implementó el control de una lavadora automática mediante una máquina de estados. En el Problema B se desarrolló un contador digital controlado mediante tres pulsadores y visualizado en un display de siete segmentos. En el Problema C se implementó un sistema de selección y ejecución de ocho secuencias luminosas utilizando ocho LEDs. Finalmente, en el Problema D se desarrolló una unidad aritmético-lógica de cuatro bits capaz de ejecutar ocho operaciones diferentes y generar las banderas Carry, Negative y Zero.

Cada solución fue diseñada, programada y simulada previamente a su implementación física, permitiendo verificar el correcto funcionamiento de las entradas, salidas y algoritmos desarrollados.

Index Terms—ATmega328P, AVR, ensamblador, GPIO, máquina de estados, display de siete segmentos, ALU, microcontroladores.

I. INTRODUCCIÓN

El microcontrolador ATmega328P dispone de diferentes recursos que permiten desarrollar sistemas digitales de control mediante la interacción directa entre software y hardware.

En este laboratorio se trabajó principalmente con los puertos digitales de entrada y salida, utilizando lenguaje ensamblador AVR para controlar directamente los registros internos del microcontrolador.

El desarrollo de las actividades permitió trabajar con entradas digitales provenientes de pulsadores e interruptores, salidas digitales conectadas a LEDs, displays y motores, operaciones aritméticas y lógicas, instrucciones de comparación, saltos condicionales y rutinas de retardo.

A lo largo del laboratorio se resolvieron cuatro problemas de complejidad creciente, comenzando con aplicaciones sencillas de entrada y salida digital y avanzando hacia sistemas secuenciales y operaciones aritmético-lógicas.

II. OBJETIVOS

El objetivo general del laboratorio fue familiarizarse con los aspectos fundamentales del microcontrolador ATmega328P y

con la programación utilizando lenguaje ensamblador AVR.

Los principales objetivos específicos fueron:

- Configurar los puertos GPIO del ATmega328P.
- Trabajar con registros de propósito general.
- Utilizar el registro de estado SREG.
- Implementar lectura de pulsadores e interruptores.
- Controlar LEDs, displays y motores mediante salidas digitales.
- Utilizar operaciones aritméticas y lógicas.
- Implementar instrucciones de comparación y salto.
- Desarrollar rutinas de retardo mediante software.
- Diseñar sistemas utilizando máquinas de estados.
- Implementar técnicas de antirrebote para pulsadores.
- Simular los sistemas antes de realizar su implementación física.
- Utilizar GitHub para mantener un registro de las diferentes versiones del código.

III. METODOLOGÍA GENERAL

Para cada uno de los problemas planteados se siguió una metodología similar.

En primer lugar se realizó el análisis de los requerimientos del sistema y se definieron las entradas y salidas necesarias. Posteriormente se diseñó la lógica de funcionamiento y se realizó la asignación de los pines del ATmega328P.

Luego se desarrolló el programa correspondiente utilizando lenguaje ensamblador AVR. Los programas fueron probados inicialmente mediante simulación con el objetivo de detectar errores antes de realizar la implementación física.

Finalmente se realizaron pruebas de funcionamiento y se registraron los resultados obtenidos.

IV. PROBLEMA A – LAVADORA AUTOMÁTICA

IV-A. Descripción del problema

El primer problema consistió en desarrollar el sistema de control de una lavadora automática.

El sistema debía administrar tres procesos principales:

- Lavado.
- Centrifugado.
- Secado.

Además, debía permitir seleccionar entre tres niveles de carga:

- Carga ligera.
- Carga media.
- Carga pesada.

IV-B. Entradas y salidas

Las principales entradas utilizadas fueron:

- Pulsador de inicio.
- Pulsador de selección de carga.
- Sensor de puerta.
- Sensor de nivel de agua.

Las salidas utilizadas fueron:

- Control del motor.
- LED de listo.
- LED de lavado.
- LED de centrifugado.
- LED de secado.
- LED de fin.
- LED de carga ligera.
- LED de carga media.
- LED de carga pesada.

IV-C. Máquina de estados

Para organizar el funcionamiento de la lavadora se implementó una máquina de estados.

Los estados principales definidos fueron:

1. Listo / Espera.
2. Selección de carga.
3. Verificación de puerta.
4. Llenado.
5. Lavado.
6. Centrifugado.
7. Secado.
8. Fin.

La Fig. 1 muestra la máquina de estados desarrollada.

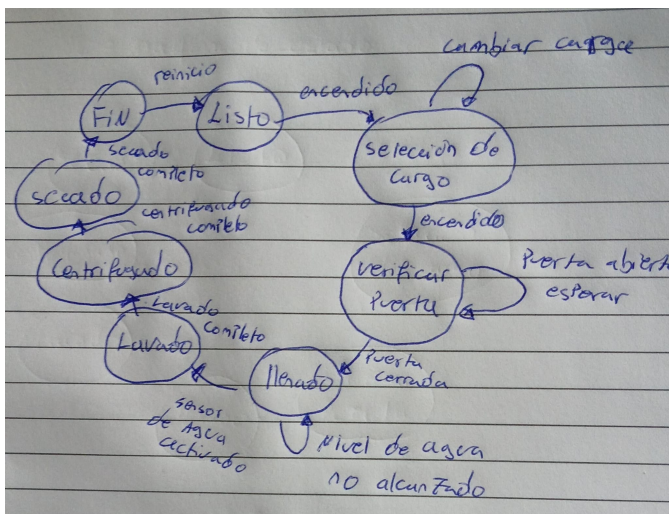


Figura 1. Máquina de estados de la lavadora automática.

IV-D. Funcionamiento

Inicialmente el sistema permanece en estado de espera.

El usuario selecciona el nivel de carga utilizando un único pulsador. Cada pulsación permite avanzar entre carga ligera, media y pesada.

Luego de presionar el pulsador de inicio, el sistema verifica el sensor de puerta. Si la puerta se encuentra abierta, la lavadora permanece esperando.

Cuando la puerta se encuentra cerrada se habilita el estado de llenado. El sistema permanece en este estado hasta que el sensor de nivel de agua indique que se alcanzó el nivel necesario.

Posteriormente se ejecutan automáticamente los procesos de lavado, centrifugado y secado.

IV-E. Proceso de lavado

Durante el proceso de lavado el tambor gira durante un determinado período de tiempo, se detiene y posteriormente vuelve a girar.

Para la carga ligera se definió:

- Giro: 2 segundos.
- Pausa: 1 segundo.
- Repeticiones: 5.

Los tiempos son incrementados según el nivel de carga seleccionado.

IV-F. Proceso de centrifugado

Durante el centrifugado el motor gira continuamente a máxima velocidad.

El tiempo base establecido es de 15 segundos y se incrementa según el nivel de carga.

IV-G. Proceso de secado

El proceso de secado se compone de:

1. Giro hacia la derecha.
2. Pausa.
3. Giro hacia la izquierda.

Los tiempos de cada etapa son ajustados según el tipo de carga seleccionado.

IV-H. Implementación en ensamblador

El programa utiliza diferentes estados representados mediante etiquetas y saltos condicionales.

Un ejemplo de la estructura general del programa es:

```

1 LISTO:
2 ; Esperar seleccion e inicio
3 rjmp LISTO
4
5
6 LAVADO:
7 ; Activar LED de lavado
8 ; Controlar motor
9 ; Ejecutar retardos
10 rjmp CENTRIFUGADO
11
12 CENTRIFUGADO:
13 ; Activar motor
14 ; Ejecutar tiempo correspondiente
15 rjmp SECADO
16
17 SECADO:

```

```

18 ; Giro derecha
19 ; Pausa
20 ; Giro izquierda
21 rjmp FIN
22
23 FIN:
24 ; Detener motor
25 ; Activar LED fin
26 rjmp LISTO

```

IV-I. Simulación

La simulación permitió verificar el funcionamiento de:

- Selección de carga.
- Pulsador de inicio.
- Sensor de puerta.
- Sensor de nivel de agua.
- LEDs indicadores.
- Control del motor.
- Transiciones entre estados.

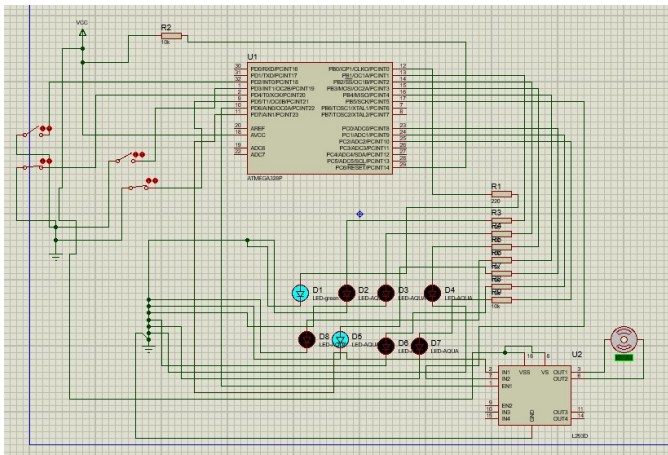


Figura 2. Simulación del Problema A. En Proteus

IV-J. Resultados

Durante las pruebas se verificó que la secuencia de funcionamiento se ejecutara correctamente.

El sistema no permite iniciar el lavado mientras la puerta se encuentra abierta y espera que se alcance el nivel correcto de agua antes de comenzar el ciclo.

También se verificó la correcta activación de los LEDs indicadores correspondientes a cada estado del proceso.

V. PROBLEMA B – CONTADOR DIGITAL

V-A. Descripción del problema

El segundo problema consistió en implementar un contador digital utilizando tres pulsadores y un display de siete segmentos.

Los pulsadores permiten:

- Incrementar el contador.
- Decrementar el contador.
- Reiniciar el contador.

El valor actual debe visualizarse permanentemente en el display.

V-B. Funcionamiento

El contador trabaja con valores comprendidos entre 0 y 9. Cuando se presiona el botón de incremento se aumenta el valor del contador.

Cuando se presiona el botón de decremento se reduce el valor.

El tercer botón reinicia el contador al valor cero.

Se implementó además una rutina de antirrebote para evitar que una única pulsación sea interpretada varias veces.

V-C. Display de siete segmentos

Para representar los números se utiliza una tabla de patrones correspondiente a los segmentos:

a, b, c, d, e, f, g

Cada número requiere una combinación específica de segmentos.

Cuadro I
PATRONES BÁSICOS DEL DISPLAY DE SIETE SEGMENTOS.

Número	Segmentos encendidos
0	a,b,c,d,e,f
1	b,c
2	a,b,d,e,g
3	a,b,c,d,g
4	b,c,f,g
5	a,c,d,f,g
6	a,c,d,e,f,g
7	a,b,c
8	a,b,c,d,e,f,g
9	a,b,c,d,f,g

V-D. Implementación en ensamblador

El valor del contador es almacenado en un registro.

Luego se determina el patrón correspondiente y se envía al puerto conectado al display.

La estructura general utilizada fue:

```

1 BUCLE:
2
3 ; Mostrar numero actual
4
5 ; Leer boton +
6 ; Si esta presionado:
7 ; incrementar contador
8
9
10 ; Leer boton -
11 ; Si esta presionado:
12 ; decrementar contador
13
14 ; Leer boton reset
15 ; Si esta presionado:
16 ; contador = 0
17
18 rjmp BUCLE

```

V-E. Antirrebote

Para evitar lecturas múltiples causadas por el rebote mecánico de los pulsadores se utilizó un pequeño retardo luego de detectar una pulsación.

Además, el programa espera que el botón sea liberado antes de aceptar una nueva pulsación.

V-F. Simulación

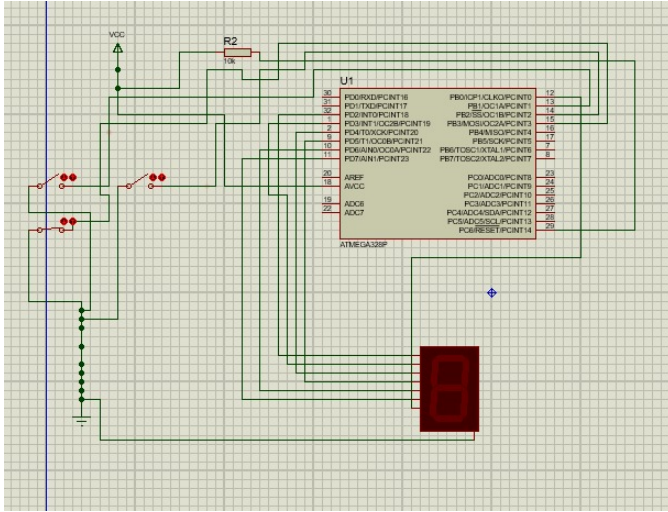


Figura 3. Simulación del contador digital.

Durante la simulación se verificaron:

- Incremento del contador.
- Decremento del contador.
- Reinicio.
- Visualización de los valores.
- Funcionamiento del antirrebote.

V-G. Resultados

El contador respondió correctamente a los tres pulsadores.

La visualización mediante el display permitió representar permanentemente el valor almacenado.

Durante las pruebas fue necesario verificar cuidadosamente la correspondencia entre los segmentos del display y los pines del microcontrolador, debido a que una conexión incorrecta produce números representados de manera incorrecta.

VI. PROBLEMA C – SELECCIÓN Y EJECUCIÓN DE SECUENCIAS

VI-A. Descripción del problema

El tercer problema consistió en implementar ocho secuencias luminosas utilizando ocho LEDs.

El sistema posee tres pulsadores.

- Botón 1: avanzar a la siguiente secuencia.
- Botón 2: regresar a la secuencia anterior.
- Botón 3: regresar directamente a la secuencia 1.

La secuencia seleccionada se ejecuta continuamente hasta que el usuario selecciona otra.

VI-B. Secuencias implementadas

Se desarrollaron ocho patrones diferentes.

Por ejemplo:

1. Desplazamiento de izquierda a derecha.
2. Desplazamiento de derecha a izquierda.
3. Encendido desde los extremos hacia el centro.
4. Encendido desde el centro hacia los extremos.

5. LEDs alternados.
6. Parpadeo simultáneo de todos los LEDs.
7. Encendido acumulativo.
8. Efecto de rebote.

VI-C. Operaciones utilizadas

Este problema permitió utilizar principalmente operaciones de desplazamiento y manipulación de bits.

Entre las instrucciones empleadas se encuentran:

- LSL: desplazamiento hacia la izquierda.
- LSR: desplazamiento hacia la derecha.
- ROL: rotación hacia la izquierda.
- ROR: rotación hacia la derecha.
- AND: aplicación de máscaras.
- ORI: activación de determinados bits.

VI-D. Implementación en ensamblador

La secuencia actual se almacena en un registro.

Dependiendo de su valor, el programa salta a la rutina correspondiente.

```
1 SELECCIONAR_SECUENCIA:
2
3
4     cpi r17, 1
5     brq SECUENCIA1
6
7     cpi r17, 2
8     brq SECUENCIA2
9
10    cpi r17, 3
11    brq SECUENCIA3
12
13    ; Continuar hasta secuencia 8
```

Por ejemplo, una secuencia de desplazamiento puede realizarse de la siguiente forma:

```
1 SECUENCIA1:
2
3
4     ldi r16, 0b00000001
5
6 LOOP_SEC1:
7
8     out PORTD, r16
9
10    rcall DELAY
11
12    lsl r16
13
14    brne LOOP_SEC1
15
16    ldi r16, 0b00000001
17
18    rjmp LOOP_SEC1
```

VI-E. Simulación

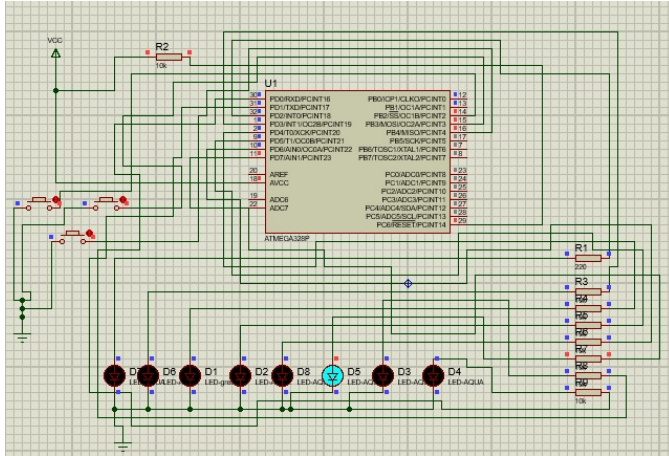


Figura 4. Simulación del sistema de ocho secuencias luminosas.

VI-F. Resultados

Durante las pruebas se verificó que cada botón realizara correctamente su función.

El sistema permitió desplazarse entre las ocho secuencias y regresar directamente a la primera.

Las operaciones de desplazamiento y rotación permitieron generar los diferentes patrones sin necesidad de almacenar una gran cantidad de valores en memoria.

VII. PROBLEMA D – UNIDAD ARITMÉTICO-LÓGICA

VII-A. Descripción del problema

El cuarto problema consistió en diseñar una Unidad Aritmético-Lógica de cuatro bits utilizando el ATmega328P.

La ALU recibe dos operandos:

$$A = A_3A_2A_1A_0$$

$$B = B_3B_2B_1B_0$$

y una señal de selección:

$$S = S_2S_1S_0$$

Dependiendo de la selección se ejecuta una operación diferente.

VII-B. Operaciones implementadas

Cuadro II
OPERACIONES DE LA ALU.

S2	S1	S0	Operación	Resultado
000			CLEAR	0000
001			SUB	A-B
010			ADD	A+B
011			XOR	A XOR B
100			AND	A AND B
101			OR	A OR B
110			SHL	A « 1
111			INC	A+1

VII-C. Selección de operación

El valor de selección es comparado con los ocho códigos posibles.

```

1 SELECCION:
2
3
4 cpi r18, 0
5 breq OP_CLEAR
6
7 cpi r18, 1
8 breq OP_SUB
9
10 cpi r18, 2
11 breq OP_ADD
12
13 cpi r18, 3
14 breq OP_XOR
15
16 cpi r18, 4
17 breq OP_AND
18
19 cpi r18, 5
20 breq OP_OR
21
22 cpi r18, 6
23 breq OP_SHL
24
25 cpi r18, 7
26 breq OP_INC

```

VII-D. Banderas de estado

Además del resultado de cuatro bits, la ALU presenta tres banderas:

- Carry (C).
- Negative (N).
- Zero (Z).

La bandera Zero se activa cuando:

$$F = 0000$$

La bandera Negative se obtiene utilizando el bit más significativo del resultado de cuatro bits:

$$N = F_3$$

La bandera Carry indica que una operación aritmética produjo un resultado superior a la capacidad de cuatro bits.

VII-E. Consideración sobre SREG

El ATmega328P trabaja internamente con registros de ocho bits.

Por esta razón, debe tenerse especial cuidado al utilizar directamente las banderas del registro SREG para una ALU de solamente cuatro bits.

Por ejemplo:

$$1111 + 0001 = 1\ 0000$$

Para una ALU de cuatro bits existe un acarreo.

Sin embargo, para el procesador AVR el resultado corresponde simplemente al valor:

$$00010000$$

y no representa un overflow de ocho bits.

Por este motivo fue necesario determinar manualmente el Carry correspondiente al bit 4 antes de limitar el resultado a cuatro bits.

Posteriormente se utiliza una máscara:

```
1
2 andi r16, 0b00001111
```

para conservar únicamente los cuatro bits del resultado.

VII-F. Salida de resultados

El resultado de la ALU se representa mediante cuatro LEDs:

$$F_3, F_2, F_1, F_0$$

También se utilizan LEDs independientes para indicar las banderas:

$$C, N, Z$$

Las entradas A, B y S pueden implementarse mediante interruptores o DIP switches.

VII-G. Simulación

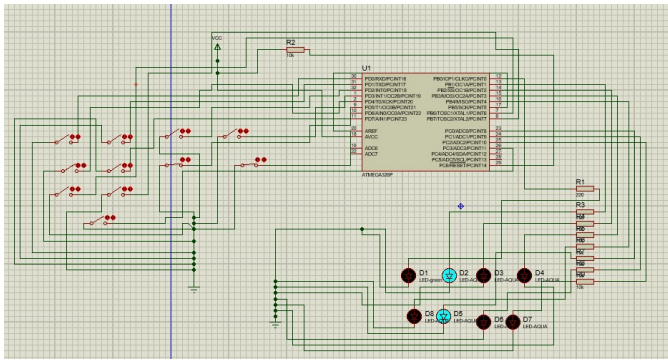


Figura 5. Simulación de la ALU de cuatro bits.

VII-H. Resultados

Se probaron diferentes combinaciones de los operandos A y B para cada una de las ocho operaciones disponibles.

Se verificó que el resultado F correspondiera con la operación seleccionada mediante S2, S1 y S0.

También se comprobaron las banderas Carry, Negative y Zero utilizando casos especialmente seleccionados.

Por ejemplo, para:

$$A = 1111$$

$$B = 0001$$

y operación suma, se obtiene:

$$F = 0000$$

con:

$$C = 1$$

$$Z = 1$$

VIII. RUTINAS COMUNES

Los diferentes problemas reutilizan determinadas estructuras y rutinas.

Entre ellas se encuentran:

- Configuración de GPIO.
- Lectura de pulsadores.
- Antirrebote.
- Rutinas de retardo.
- Comparaciones.
- Saltos condicionales.
- Manipulación de bits.

VIII-A. Rutina de retardo

Un ejemplo simplificado de rutina de retardo es:

```
1
2 DELAY:
3
4 ldi r20, 255
5
6 DELAY_LOOP:
7
8 dec r20
9 brne DELAY_LOOP
10
11 ret
```

Para generar tiempos mayores se utilizaron múltiples ciclos anidados.

IX. CONTROL DE VERSIONES CON GITHUB

Los códigos desarrollados fueron almacenados en un repositorio público de GitHub.

El control de versiones permitió registrar diferentes etapas del desarrollo.

Se realizaron como mínimo tres modificaciones significativas.

IX-A. Primera versión

Se implementó la configuración inicial de los registros y de los puertos de entrada y salida.

IX-B. Segunda versión

Se incorporó la lógica principal de funcionamiento correspondiente a cada problema.

IX-C. Tercera versión

Se realizaron correcciones luego de las simulaciones y se incorporaron mejoras generales.

El repositorio se encuentra disponible en:

[https:](https://github.com/Braian-Arias/Tec.Micro/tree/main/Laboratorio1)

[//github.com/Braian-Arias/Tec.Micro/tree/main/Laboratorio1](https://github.com/Braian-Arias/Tec.Micro/tree/main/Laboratorio1)

X. RESULTADOS GENERALES

Los cuatro problemas permitieron comprobar el funcionamiento de diferentes recursos del microcontrolador ATmega328P.

En el Problema A se logró implementar un sistema secuencial mediante una máquina de estados.

En el Problema B se trabajó con entradas digitales y la representación de información mediante un display de siete segmentos.

En el Problema C se utilizaron operaciones de desplazamiento y manipulación de bits para generar diferentes secuencias luminosas.

Finalmente, en el Problema D se implementaron operaciones aritméticas y lógicas.

Las simulaciones permitieron detectar y corregir diferentes errores antes de realizar las implementaciones físicas.

XI. CONCLUSIONES

La realización del Laboratorio 1 permitió comprender de manera práctica el funcionamiento de los puertos de entrada y salida del microcontrolador ATmega328P y su control mediante lenguaje ensamblador.

Los diferentes problemas planteados permitieron avanzar progresivamente desde el control básico de entradas y salidas hasta sistemas de mayor complejidad.

La implementación de la lavadora permitió trabajar con máquinas de estados y control secuencial. El contador digital permitió estudiar la representación de información utilizando un display de siete segmentos y el tratamiento de pulsadores.

Las secuencias luminosas permitieron aplicar operaciones de desplazamiento, máscaras y manipulación directa de bits, mientras que el desarrollo de la ALU permitió profundizar en las operaciones aritméticas y lógicas y en el funcionamiento de las banderas de estado.

El uso de lenguaje ensamblador permitió observar de manera directa cómo cada instrucción modifica registros, puertos y banderas internas del microcontrolador.

Finalmente, la simulación previa y el control de versiones mediante GitHub fueron herramientas importantes para identificar errores, registrar cambios y mantener una trazabilidad del proceso de desarrollo.

REFERENCIAS

- [1] Material recibido en clase Curso Tecnología de Microprocesamiento
- [2] Microchip Technology Inc., "ATmega328P 8-bit AVR Microcontroller Datasheet."
- [3] Microchip Technology Inc., "AVR Instruction Set Manual."
- [4] Arduino, "Arduino Uno Rev3 Documentation."